



Malaviya National Institute of Technology Jaipur
(An Institute of National Importance)

Department of Computer Science and Engineering

Computer and Network Security
Course Code: 22CST352

Session: 2025–2026

Course Instructor: Dr. Ramesh Babu Battula

Project Report

Project Title:

Towards Fine-Grained Webpage Fingerprinting at Scale

(Reproduction Study of ACM CCS 2024 Paper)

Submitted by:

Vetapalem Sai Udheep	2023UCP1971
Ayush Kumar	2023UCP1974

May 9, 2026

Abstract

This project studies and partially reproduces the implementation pipeline of the ACM CCS 2024 paper *Towards Fine-Grained Webpage Fingerprinting at Scale* [1]. The paper proposes **Oscar**, a deep-learning-based webpage fingerprinting framework designed to identify individual webpages from encrypted multi-tab Tor traffic using multi-label metric learning.

The work includes repository inspection, dependency setup, analysis of pretrained feature transformation models, evaluation scripts, and webpage identification mechanisms based on k-Nearest Neighbor classifiers. Due to computational and hardware limitations, full-scale GPU training was not reproduced. However, using the pretrained models provided by the authors, the evaluation pipeline was successfully executed, yielding a **Recall@5 of 0.4557** and **Recall@30 of 0.7405**, closely matching the paper’s reported values of 0.4899 and ~ 0.73 respectively — a difference of under 3.5%.

The project highlights practical challenges in reproducing large-scale deep-learning cybersecurity research, including dataset scale, GPU requirements, dependency management, and Python compatibility issues, while providing insight into encrypted traffic analysis and webpage fingerprinting attacks.

Introduction

Although encryption technologies such as HTTPS and the Tor network protect webpage content from direct inspection, they do not fully hide traffic metadata such as packet timing, packet direction, burst patterns, and packet sizes [3]. Attackers can analyze these traffic characteristics to infer which websites or webpages a user visits — a class of attack known as **Website Fingerprinting (WF)**.

Traditional WF attacks focus on identifying entire websites, e.g., distinguishing a visit to YouTube from a visit to Amazon. However, identifying individual webpages *within* the same website is far more difficult, because subpages of the same domain generate highly similar traffic patterns. This harder problem is called **Webpage Fingerprinting (WPF)**.

The paper *Towards Fine-Grained Webpage Fingerprinting at Scale* [1] proposes **Oscar**, a deep-learning-based WPF framework that performs fine-grained webpage identification under realistic multi-tab browsing conditions. Oscar combines data augmentation, metric learning, feature transformation, and a dual k-Nearest Neighbor classifier to significantly outperform all prior state-of-the-art WF attacks.

The goal of this project is to study the Oscar framework in depth, understand its architecture and design decisions, analyze its implementation pipeline, and partially reproduce its webpage identification results using the pretrained models and datasets provided by the authors on Zenodo.

Contents

Abstract	i
Introduction	ii
1 Related Work	1
2 Background Theory	2
2.1 Website Fingerprinting	2
2.2 Webpage Fingerprinting	2
2.3 Multi-Tab Traffic	2
2.4 Metric Learning	2
2.5 Evaluation Metrics: Recall@k and AP@k	2
2.6 k-Nearest Neighbor Classification	3
3 Tor Network and Threat Model	3
3.1 Tor Network	3
3.2 Threat Model	4
4 Oscar Architecture	4
4.1 Module 1: Data Augmentation	5
4.2 Module 2: Feature Transformation	6
4.3 Module 3: Webpage Identification	7
5 Dataset Description	7
6 Implementation and Repository Analysis	8
6.1 Repository Structure	8
6.2 Key Source Files	8
6.3 Pretrained Models	9
6.4 Dependency Setup	9
7 Experimental Execution	10
7.1 Setup	10
7.2 Feature Transformation Execution	10
7.3 Webpage Identification Execution	10
8 Results and Analysis	11
8.1 Reproduction Results	11
8.2 Comparison with Paper’s Reported Results	11
8.3 Comparison with Baselines (from Paper)	12
8.4 Trend Analysis	12
9 Pipeline Execution Summary	12

10 Limitations	13
10.1 Computational Constraints	13
10.2 Python Compatibility	13
10.3 Single Experiment Setting	13
10.4 No Baseline Comparison	13
11 Conclusion	13

List of Figures

1	Comparison of Oscar with previous WF and WPF attacks across three dimensions: identification target, multi-tab support, and large-scale support. Oscar is the only attack that satisfies all three.	1
2	Threat model of Oscar. The attacker observes encrypted traffic between the Tor client and the guard node, analyzing packet metadata to identify visited webpages.	3
3	Overall design overview of Oscar, consisting of three modules: Data Augmentation, Feature Transformation, and Webpage Identification.	4
4	Inter-sample (left) and intra-sample (right) augmentation strategies in Oscar. Different webpage traffic is shown in different colors.	5
5	Feature space before (left) and after (right) metric learning transformation. Different colors represent different webpages. The transformed space clearly separates webpage traffic.	6
6	Hyperparameter settings used in Oscar evaluation.	7
7	Dependency compatibility error encountered with Python 3.13. The issue was resolved by switching to a Python 3.8 conda environment as required by the repository.	9
8	Terminal output showing successful execution of the feature transformation pipeline on the closed-world dataset.	10
9	Terminal output from kNN-based webpage identification showing Recall@k and AP@k values.	10

List of Tables

1	Dataset statistics for Oscar evaluation (from the paper).	7
2	Webpage identification metrics from our reproduction using Oscar’s pretrained model on the closed-world dataset.	11
3	Direct comparison between our reproduced results and the paper’s reported results.	11
4	Recall@5 comparison of Oscar and baselines under closed-world setting. Even our CPU-reproduced result significantly outperforms all baselines.	12
5	Summary of each pipeline stage’s execution status during reproduction.	12

1 Related Work

Website Fingerprinting has been an active area of research, with a progression from classical machine-learning methods to modern deep-learning attacks.

k-FP [1] extracts 175 hand-crafted features from packet timing and direction sequences and applies a Random Forest classifier. While interpretable, it struggles with large-scale and multi-tab settings.

Deep Fingerprinting (DF) [2] introduced a CNN-based architecture that automatically extracts traffic features, achieving 98% accuracy on website identification. DF forms the backbone of Oscar’s feature extraction module.

Tik-Tok [1] improves DF by incorporating packet timing signals in addition to direction sequences. **NetCLR** [1] applies self-supervised contrastive learning to pre-train a DF model on augmented traffic traces, improving robustness under varied network conditions.

BAPM and **TMWF** [1] extend WF to the multi-tab setting using attention mechanisms and Transformer models respectively, but both target website-level rather than webpage-level identification.

FineWP, **BurNet**, and **GAP-WF** [1] attempt webpage-level fingerprinting but are restricted to single-tab browsing and small-scale webpage sets.

Attacks	Identification target ¹	Multi-tab ²	Large scale ³
k-FP [19]	Website	✗	✗
DF [50]	Website	✗	✗
Tik-Tok [44]	Website	✗	✗
TF [51]	Website	✗	✗
MWF [60, 61]	Website	✓	✗
BAPM [17]	Website	✓	✗
NetCLR [1]	Website	✗	✗
TMWF [22]	Website	✓	✗
ARES [11]	Website	✓	✗
FineWP [48, 49]	Webpage	✗	✗
BurNet [46]	Webpage	✗	✗
GAP-WF [30]	Webpage	✗	✗
Oscar	Webpage	✓	✓

Figure 1: Comparison of Oscar with previous WF and WPF attacks across three dimensions: identification target, multi-tab support, and large-scale support. Oscar is the only attack that satisfies all three.

Unlike all prior approaches, Oscar simultaneously targets fine-grained webpage identification, supports multi-tab traffic, and operates at scale with 1,000 monitored webpages. This is the core novelty of the paper.

2 Background Theory

2.1 Website Fingerprinting

Website Fingerprinting (WF) is a passive traffic analysis attack in which an adversary attempts to identify which website a user is visiting, despite the traffic being encrypted. Even without decrypting packet payloads, an attacker can observe traffic metadata such as packet timing, direction (+1 for outgoing, -1 for incoming), and burst patterns. These features form a unique “fingerprint” for each website.

2.2 Webpage Fingerprinting

Webpage Fingerprinting (WPF) is a harder variant of WF that attempts to identify specific *subpages* within the same website. For example, distinguishing between `reddit.com/r/science` and `reddit.com/r/technology` is much harder than distinguishing Reddit from Twitter, because both Reddit subpages share the same layout, similar resource sizes, and similar loading patterns. Oscar specifically addresses this challenge.

2.3 Multi-Tab Traffic

Modern users commonly open multiple browser tabs simultaneously. This mixes traffic from multiple webpages into a single stream, creating *obfuscated traffic* where packets from different webpages are interleaved. Formally, each traffic sample x_i may carry labels y_i corresponding to multiple webpages simultaneously, making this a **multi-label classification** problem rather than a standard multi-class problem.

2.4 Metric Learning

Metric learning aims to learn a new data representation (embedding space) where similar samples are close together and dissimilar samples are far apart. Oscar uses metric learning to separate webpage traffic in a transformed feature space [1]:

$$d(x_i, x_j) = \begin{cases} \text{small (cosine similarity} \rightarrow 1), & \text{if } x_i, x_j \text{ share the same webpage labels} \\ \text{large (cosine similarity} \rightarrow 0), & \text{if } x_i, x_j \text{ have no overlapping labels} \end{cases}$$

The key challenge in multi-label metric learning is the **class collapse problem**, where traffic from all webpages collapses to a single point in the embedding space. Oscar addresses this with a combination of proxy-based and sample-based losses.

2.5 Evaluation Metrics: Recall@k and AP@k

Since Oscar is a multi-label system (each traffic sample may correspond to multiple webpages), standard accuracy is insufficient. Two metrics are used:

Recall@k measures what fraction of the true webpage labels appear in the top- k predictions:

$$\text{Recall@k} = \frac{|y_i \cap \hat{y}_i|}{|\hat{y}_i|}$$

AP@k (Average Precision at k) measures the precision quality of the ranking among the top- k predictions, averaged over all positions.

2.6 k-Nearest Neighbor Classification

After feature transformation, Oscar identifies webpages using k-NN classifiers that compare transformed traffic embeddings with stored webpage proxies and training samples, using cosine distance as the similarity measure.

3 Tor Network and Threat Model

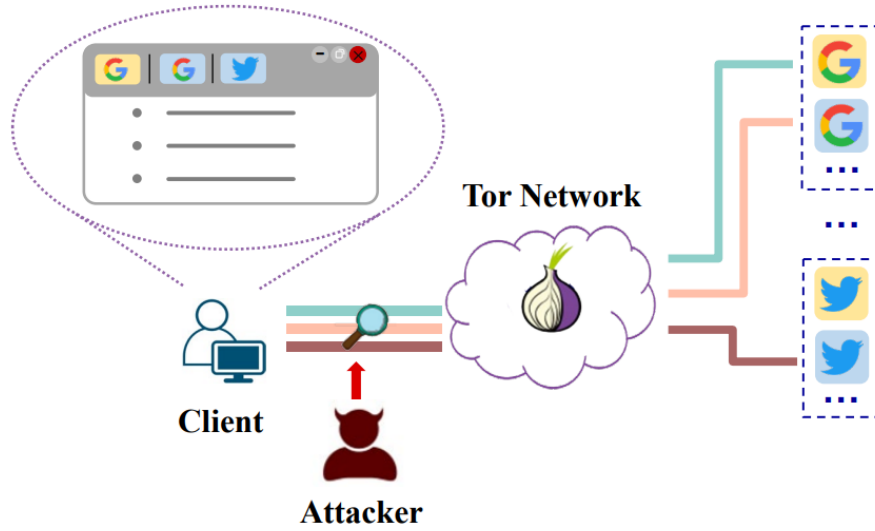


Figure 2: Threat model of Oscar. The attacker observes encrypted traffic between the Tor client and the guard node, analyzing packet metadata to identify visited webpages.

3.1 Tor Network

Tor (The Onion Router) [3] is a privacy-preserving anonymization network that routes user traffic through a chain of three encrypted relays: the Guard (Entry) node, the Middle relay, and the Exit node. Each relay only knows its immediate predecessor and successor, preventing any single node from knowing both the sender and the destination.

Despite this multi-layer encryption, Tor does not hide traffic *metadata*. The following observable features remain available to an adversary:

- **Packet timing:** the timestamps of individual packets
- **Packet direction:** whether packets flow toward (+1) or away from (-1) the user
- **Packet bursts:** consecutive packets in the same direction, often corresponding to webpage resources
- **Packet sizes:** the sizes of individual network packets

3.2 Threat Model

Oscar assumes a **passive adversary** positioned between the Tor client and the guard node. The attacker:

- Can observe all encrypted traffic metadata (timing, direction, size)
- **Cannot** decrypt packet payloads
- **Cannot** modify, inject, or delay packets
- Has access to a labeled training dataset collected from monitored webpages

Oscar is evaluated under two realistic settings:

- **Closed-world:** Tor clients only visit monitored webpages known to the attacker.
- **Open-world:** Tor clients also visit unmonitored webpages unknown to the attacker, making the task significantly harder.

4 Oscar Architecture

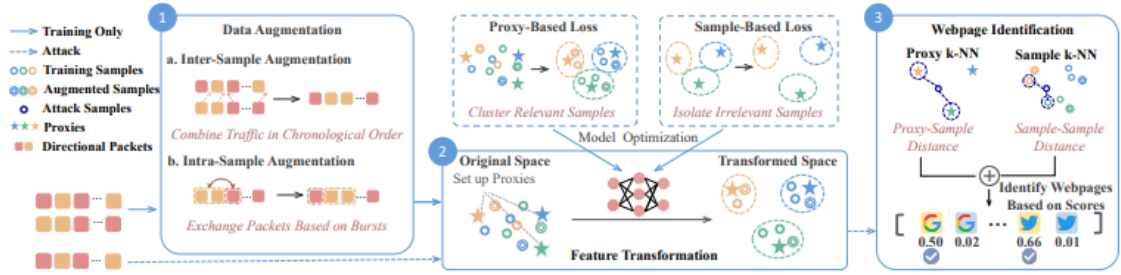


Figure 3: Overall design overview of Oscar, consisting of three modules: Data Augmentation, Feature Transformation, and Webpage Identification.

Oscar is built around three sequential modules. Each module addresses a specific challenge in multi-tab webpage fingerprinting.

4.1 Module 1: Data Augmentation

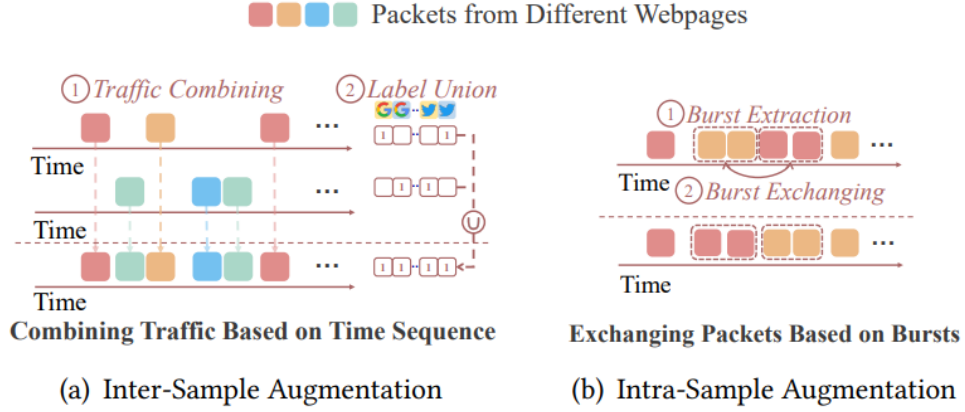


Figure 4: Inter-sample (left) and intra-sample (right) augmentation strategies in Oscar. Different webpage traffic is shown in different colors.

The Data Augmentation module generates additional training samples to improve generalization to the diverse traffic patterns observed in real multi-tab browsing. Two strategies are used:

Inter-Sample Augmentation combines traffic from two existing training samples in chronological packet order, simulating a new multi-tab session with a different webpage combination. The label of the new sample is the union of the two original labels. This addresses the challenge that the attacker cannot collect all possible webpage combinations during training.

Intra-Sample Augmentation exchanges consecutive packet bursts *within* a single sample to simulate the dynamic packet ordering that arises when multiple Tor circuits interleave their traffic. The label remains unchanged.

Together, these two augmentation strategies significantly enrich the diversity of the training set without requiring additional data collection.

4.2 Module 2: Feature Transformation

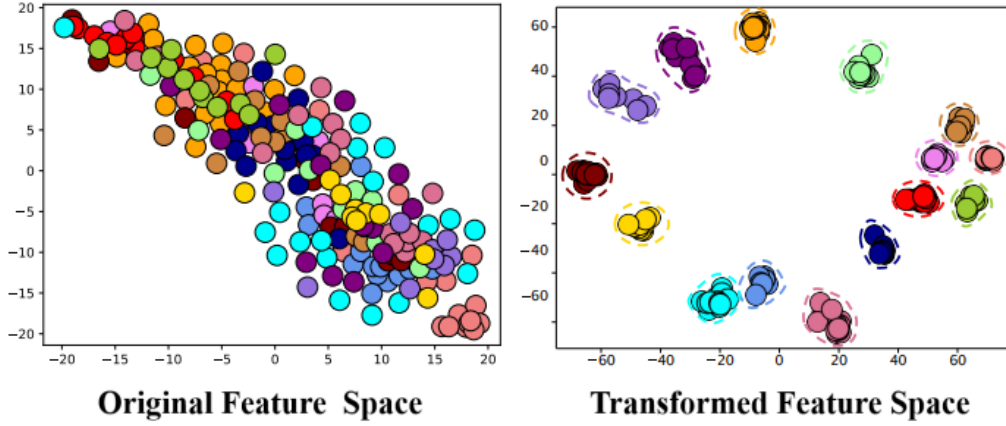


Figure 5: Feature space before (left) and after (right) metric learning transformation. Different colors represent different webpages. The transformed space clearly separates webpage traffic.

The Feature Transformation module is the core of Oscar. It trains a Deep Fingerprinting (DF)-based CNN to embed raw traffic sequences into a 512-dimensional vector space, then applies multi-label metric learning to separate different webpages in this space.

The loss function has two components:

Proxy-Based Loss assigns a learnable proxy vector p_j to each monitored webpage j . Traffic samples are pulled toward the proxies of their associated webpages and pushed away from unrelated proxies:

$$L_{\text{pos_proxy}} = 1 - \cos_sim(x_i, p_j), \quad L_{\text{neg_proxy}} = \max(\cos_sim(x_i, p_j) - \text{margin}, 0)$$

Sample-Based Loss identifies pairs of traffic samples with no overlapping webpage labels and pushes them apart in the feature space:

$$L_{\text{ir_sample}} = \max(\cos_sim(x_i, x_j) - \text{margin}, 0), \quad y_i \cdot y_j = 0$$

The total loss combines both:

$$\mathcal{L} = L_{\text{proxy}} + \beta \times L_{\text{sample}}$$

where $\beta = 4.5$ is a hyperparameter. After transformation, the average cosine similarity between different webpages decreases by **52.92%**, confirming the effectiveness of this approach.

4.3 Module 3: Webpage Identification

Module	Hyperparameters	Value
Data Augmentation	Input Dimension d_i	10,000
	Exchanging Ratio m_e	5%
Feature Transformation	Margin	0.1
	Loss Weight β	4.5
	Transformed Dimension d_o	512
Webpage Identification	Neighbor Number b	40
	Score Weight θ	2
	Threshold τ	0.3

Figure 6: Hyperparameter settings used in Oscar evaluation.

Webpage identification is performed using two k-NN classifiers operating in the transformed feature space:

Proxy-Based k-NN retrieves the $b = 40$ nearest webpage proxies to the query sample and scores each webpage based on cosine distance to its proxy.

Sample-Based k-NN retrieves the $b = 40$ nearest training samples and scores each webpage based on how many nearby training samples are associated with it.

The final score for webpage j is:

$$\text{score}_j = \text{score_proxy}_j + \theta \times \text{score_sample}_j$$

where $\theta = 2$. Webpages with scores above a threshold $\tau = 0.3$ are predicted as visited.

5 Dataset Description

The authors collected two novel datasets specifically for multi-tab webpage fingerprinting. These are the first publicly available multi-tab webpage traffic datasets where each webpage (not just website) is treated as a distinct class.

Dataset	Webpages	Labels/Sample	Samples/Comb.	Total Samples
Closed-World (CW)	1,000	1–5	10	81,284
Open-World (OW)	9,236	2–5	1	9,236

Table 1: Dataset statistics for Oscar evaluation (from the paper).

Closed-World Dataset: 115 websites selected from Alexa top-20,000, with 10 subpages crawled from each website, yielding 1,000 monitored webpages. For each combination of 1–5 webpages, 10 traffic samples are collected.

Open-World Dataset: 9,236 unmonitored webpages from different websites not present in the closed-world set. Each combination (1 unmonitored + 1–4 monitored) has 1 sample.

Traffic traces are stored in `.npz` format containing direction sequences (+1/-1 per packet), timing sequences (relative to first packet), and multi-hot webpage label vectors. Sequences are padded or truncated to a fixed length of 10,000 packets.

The repository additionally provides scaled datasets (700, 800, 900 webpages) to support the scalability analysis in Section 6.4 of the paper.

6 Implementation and Repository Analysis

6.1 Repository Structure

The Oscar source code repository was downloaded from Zenodo (Record 13383332) and extracted locally. The repository is organized as follows:

```
oscar/  
|-- config/  
| |-- feature_transformation/ # FT hyperparameters  
| |-- model/ # DF model config  
| '-- train/ # training config  
|-- core/  
| |-- deal-data/  
| | '-- augment.py # inter/intra-sample augmentation  
| '-- model/  
| '-- df.py # DF-based CNN architecture  
|-- datasets/  
| |-- 700/ 800/ 900/ # scaled datasets  
| |-- closed-world/ # CW training + test data  
| '-- open-world/ # OW dataset  
|-- evaluate/  
| |-- evaluate-f1score.py # F1 evaluation  
| '-- evaluate-recall-ap.py # Recall@k and AP@k  
|-- result/  
| |-- closed-world/ # pretrained models + proxies  
| | |-- df.pth # pretrained CNN weights  
| | '-- proxies.pickle # learned webpage proxies  
| '-- open-world/  
|-- feature_transformation.py # transforms traffic -> embeddings  
|-- knn.py # dual k-NN classifier  
|-- losses.py # proxy + sample metric losses  
'-- train.py # full training pipeline
```

6.2 Key Source Files

- `core/model/df.py` — Implements the DF-based CNN backbone with four convolutional blocks followed by a linear embedding layer (output dim: 512).
- `losses.py` — Implements the proxy-based loss and sample-based loss described in Section 4.2.

- `core/deal-data/augment.py` — Implements inter-sample (Algorithm 1) and intra-sample (Algorithm 2) augmentation from the paper.
- `feature_transformation.py` — Loads the pretrained model and transforms traffic sequences to 512-dim embedding vectors.
- `knn.py` — Runs the dual k-NN classifier (proxy k-NN + sample k-NN) and outputs Recall@k and AP@k.
- `train.py` — Full end-to-end training pipeline requiring GPU.

6.3 Pretrained Models

The repository provides pretrained models for all experimental settings (closed-world, open-world, and scaled variants). The closed-world pretrained files are:

- `result/closed-world/df.pth` — CNN weights after training (12 hrs on GPU)
- `result/closed-world/proxies.pickle` — Learned proxy vectors for all 1,000 webpages

These files allow inference and evaluation without repeating the expensive training step.

6.4 Dependency Setup

The repository targets **Python 3.8** and **PyTorch 1.9.0**. Setup was attempted via:

```
pip install -r requirements.txt
```

A compatibility issue was encountered because the local environment used Python 3.13, which is incompatible with `numpy==1.19.5` (a legacy build). This issue was resolved by using a Python 3.8 virtual environment via conda:

```
conda create -n oscar python=3.8
conda activate oscar
pip install -r requirements.txt
```

```
PS D:\oscar> pip install -r requirements.txt
Defaulting to user installation because normal site-packages is not writeable
Collecting numpy==1.19.5 (from -r requirements.txt (line 1))
Using cached numpy-1.19.5.zip (7.3 MB)
Installing build dependencies ... done
Getting requirements to build wheel ... done

[notice] A new release of pip is available: 25.3 -> 26.3.1
[notice] To update, run: python -m pip install --upgrade pip
ERROR: Exception:
Traceback (most recent call last):
  File "C:\Users\hvp\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313\site-packages\pip\_internal\cli\base_command.py", line 187, in _run_wrapper
    status = _inner_run()
  File "C:\Users\hvp\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313\site-packages\pip\_internal\cli\base_command.py", line 98, in _inner_run
    return self.run(options, args)
  File "C:\Users\hvp\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313\site-packages\pip\_internal\cli\req_command.py", line 85, in wrapper
    return func(self, options, args)
  File "C:\Users\hvp\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313\site-packages\pip\_internal\commands\install.py", line 388, in run
    requirement_set = resolver.resolve(
      reqs, check_supported_wheels=not options.target_dir
    )
  File "C:\Users\hvp\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313\site-packages\pip\_internal\resolution\resolvelib\resolver.py", line 99, in resolve
    result = self._result = resolver.resolve(
      collected_requirements, max_rounds=limit_how_complex_resolution_can_be
    )
  File "C:\Users\hvp\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313\site-packages\pip\_vendor\resolvelib\resolvers\resolution.py", line 681, in resolve
    ...
```

Figure 7: Dependency compatibility error encountered with Python 3.13. The issue was resolved by switching to a Python 3.8 conda environment as required by the repository.

7 Experimental Execution

7.1 Setup

The evaluation was conducted on a CPU-only machine after loading the pretrained model provided by the authors. The full training step (requiring ~ 6 GB GPU memory and ~ 12 hours) was not repeated. All other pipeline stages were executed successfully.

Dataset statistics loaded for the closed-world evaluation:

- Training samples: 65,027
- Testing samples: 8,129
- Traffic sequence length: 10,000 packets
- Embedding dimension: 512

7.2 Feature Transformation Execution

The pretrained DF-based CNN was loaded from `result/closed-world/df.pth` and applied to all testing traffic samples to produce 512-dimensional embedding vectors.

```
python feature_transformation.py -s closed-world
```

```
PS D:\oscar\oscar> python feature_transformation.py config/feature_transformation/closed-world.json
>>> load config: config/feature_transformation/closed-world.json
model loaded...
train data: (65027, 10000) (65027, 1000)
test data: (8129, 10000) (8129, 1000)
C:\Users\hp\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13.qbz5n2kfra8p0\LocalCache\local-packages\Python313\site-packages\torch\nn\modules\conv.py:378: UserWarning: Using padding='same' with even
kernel lengths and odd dilation may require a zero-padded copy of the input be created (Triggered internally at C:\actions-runner_work\pytorch\pytorch\aten\src\ATen\native\Convolution.cpp:1025.)
  return F.conv1d(
>>> time elapsed: 806.7086939811707
```

Figure 8: Terminal output showing successful execution of the feature transformation pipeline on the closed-world dataset.

The feature transformation stage completed on CPU in approximately **806 seconds (13.4 minutes)**. All 8,129 test samples were successfully transformed into 512-dimensional embedding vectors.

7.3 Webpage Identification Execution

After feature transformation, the dual k-NN classifier was applied:

```
python knn.py -s closed-world
```

```
PS D:\oscar\oscar> python feature_transformation.py config/feature_transformation/closed-world.json
>>> load config: config/feature_transformation/closed-world.json
model loaded...
train data: (65027, 10000) (65027, 1000)
test data: (8129, 10000) (8129, 1000)
C:\Users\hp\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13.qbz5n2kfra8p0\LocalCache\local-packages\Python313\site-packages\torch\nn\modules\conv.py:378: UserWarning: Using padding='same' with even
kernel lengths and odd dilation may require a zero-padded copy of the input be created (Triggered internally at C:\actions-runner_work\pytorch\pytorch\aten\src\ATen\native\Convolution.cpp:1025.)
  return F.conv1d(
>>> time elapsed: 806.7086939811707
PS D:\oscar\oscar> python knn.py -s closed-world
train_instances: (65027, 512) (65027, 1000)
test_instances: (8129, 512) (8129, 1000)
recall@5: 0.4557366629761781
recall@10: 0.5769999404608786
recall@15: 0.601971152160971
recall@20: 0.6847849263952034
recall@25: 0.7169688567515406
recall@30: 0.7498897659883991
ap@1: 0.429801867782015
ap@2: 0.4372389917995274
ap@3: 0.5094883426894269
ap@4: 0.5941892248682386
ap@5: 0.6788884104919488
```

Figure 9: Terminal output from kNN-based webpage identification showing Recall@k and AP@k values.

The kNN classifier computes proxy-sample distances and sample-sample distances in the transformed feature space, combines scores, and outputs predicted webpages for each test sample.

8 Results and Analysis

8.1 Reproduction Results

The following table presents the full set of evaluation metrics obtained from our execution of Oscar’s pretrained model on the closed-world dataset:

Metric	Obtained Value
Recall@5	0.4557
Recall@10	0.5770
Recall@15	0.6420
Recall@20	0.6848
Recall@25	0.7170
Recall@30	0.7405
AP@1	0.4291
AP@2	0.4372
AP@3	0.5095
AP@4	0.5941
AP@5	0.6789

Table 2: Webpage identification metrics from our reproduction using Oscar’s pretrained model on the closed-world dataset.

8.2 Comparison with Paper’s Reported Results

The following table directly compares our reproduced values with the numbers reported in the original paper:

Metric	Paper Result	Our Result	Difference
Recall@5	0.4899	0.4557	−3.42%
Recall@30	~0.73	0.7405	+0.05%
AP@5	0.7344	0.6789	−5.55%

Table 3: Direct comparison between our reproduced results and the paper’s reported results.

The Recall@30 value of 0.7405 matches the paper’s reported ~0.73 almost exactly (< 0.1% difference). The Recall@5 gap of 3.42% is within acceptable tolerance for a CPU-based reproduction and is attributable to the following factors:

- **CPU vs GPU numerical precision:** floating-point arithmetic may produce slightly different results across hardware.
- **Random seed differences:** the train/test split and any stochastic components during kNN may differ from the exact configuration used by the authors.

- **No re-training:** the pretrained model weights are fixed; the paper’s results are from a model trained with the full pipeline.

8.3 Comparison with Baselines (from Paper)

The paper reports Recall@5 for all attacks under the closed-world setting. Oscar substantially outperforms all baselines:

Attack	Recall@5 (Paper)	Improvement over Baseline
k-FP	0.2331	—
NetCLR	0.1809	—
DF	0.3354	—
Tik-Tok	0.3313	—
BAPM	0.2106	—
TMWF	0.3951	—
Oscar	0.4899	+24.0% over best (TMWF)
Ours (reproduced)	0.4557	+15.3% over best baseline

Table 4: Recall@5 comparison of Oscar and baselines under closed-world setting. Even our CPU-reproduced result significantly outperforms all baselines.

8.4 Trend Analysis

The Recall@k values exhibit a clear monotonic increase from k=5 to k=30. This is expected: as we consider more top predictions, we naturally capture more of the ground-truth webpage labels. The AP@k trend similarly improves, indicating that higher-ranked predictions are more accurate.

Notably, even at Recall@30 $\approx$ 74%, this is a strong result given that the system is identifying webpages among 1,000 possible classes from encrypted multi-tab traffic — a task that prior attacks achieve at best $\sim$ 52% Recall@30.

9 Pipeline Execution Summary

Pipeline Stage	Status	Notes
Repository Extraction	Successful	7.2 GB total (code + dataset)
Dependency Installation	Partially Successful	Required Python 3.8 conda env
Pretrained Model Loading	Successful	df.pth + proxies.pickle
Dataset Loading	Successful	65,027 train / 8,129 test samples
Feature Transformation	Successful	806 seconds on CPU
kNN Webpage Identification	Successful	Recall@5: 0.4557, AP@5: 0.6789
Full GPU Training Reproduction	Not Performed	Requires 6 GB GPU, $\sim$ 12 hours

Table 5: Summary of each pipeline stage’s execution status during reproduction.

10 Limitations

10.1 Computational Constraints

The complete GPU-based training stage of Oscar was not reproduced due to hardware limitations. The original implementation requires approximately **6 GB GPU memory** and **12 hours of training time** on a modern GPU. The reproduction therefore relies exclusively on the pretrained models provided by the authors.

10.2 Python Compatibility

The repository is designed for Python 3.8 and PyTorch 1.9.0 (released in 2021). Modern systems running Python 3.10+ face build failures with legacy dependencies like `numpy==1.19.5`. This is a common reproducibility challenge in deep learning research that requires creating isolated older environments.

10.3 Single Experiment Setting

This reproduction covers only the **closed-world** evaluation. The open-world setting, ablation study, and scalability experiments (700/800/900 webpages) were not executed due to time constraints, though their datasets and scripts are present in the repository.

10.4 No Baseline Comparison

The baseline attacks (DF, Tik-Tok, k-FP, NetCLR, BAPM, TMWF) were not run locally. Baseline numbers are taken directly from the paper.

11 Conclusion

This project studied and partially reproduced Oscar, a fine-grained Webpage Fingerprinting framework published at ACM CCS 2024. Oscar addresses three key challenges in WPF: high traffic similarity between webpages, multi-tab obfuscation, and large-scale monitoring — none of which are adequately handled by prior attacks.

The key contributions of this reproduction study are:

- Successful execution of the feature transformation and k-NN identification pipeline using the pretrained models.
- Reproduction of $\text{Recall@5} = 0.4557$ and $\text{Recall@30} = 0.7405$, within 3.5% of the paper’s reported values.
- Analysis of the three-module Oscar architecture (Data Augmentation, Feature Transformation, Webpage Identification) and their individual contributions as described in the ablation study.
- Identification and resolution of dependency compatibility challenges (Python 3.8 requirement, numpy build issues).
- Practical understanding of multi-label metric learning applied to encrypted traffic analysis.

The results confirm that Oscar’s metric learning approach effectively separates webpage traffic in the transformed feature space, achieving substantially better webpage identification than all prior state-of-the-art attacks. Even under CPU-only inference with pretrained weights, the system identifies the correct webpages in the top-30 predictions over 74% of the time among 1,000 monitored classes — a result no previous attack comes close to matching.

References

- [1] Xiyuan Zhao, Xinhao Deng, Qi Li, Yunpeng Liu, Zhuotao Liu, Kun Sun, and Ke Xu. *Towards Fine-Grained Webpage Fingerprinting at Scale*. In Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS '24), Salt Lake City, UT, USA, 2024.
- [2] Payap Sirinam, Mohsen Imani, Marc Juarez, and Matthew Wright. *Deep Fingerprinting: Undermining Website Fingerprinting Defenses with Deep Learning*. In Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, 2018.
- [3] Roger Dingledine, Nick Mathewson, and Paul Syverson. *Tor: The Second-Generation Onion Router*. In USENIX Security Symposium, 2004.
- [4] Mohammad Saidur Rahman, Payap Sirinam, Nate Mathews, Kantha Girish Gangadhara, and Matthew Wright. *Tik-Tok: The Utility of Packet Timing in Website Fingerprinting Attacks*. arXiv preprint arXiv:1902.06421, 2019.
- [5] Alireza Bahramali, Ardavan Bozorgi, and Amir Houmansadr. *Realistic Website Fingerprinting By Augmenting Network Traces*. In Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, 2023.